Software Project Proposal

MRAS Eco (Medical Room Automation System - Ecological Integration)

Institution: University of Vocational Technology (UOVT)

Department: Software and Intelligent Systems

Module: IT304040 Python Programming – Group Project

Project Team - 03:

- L.B. Charith Jeewan (SIS/24/B2/36)
- W.I.L. Withana (SIS/24/B2/38)
- H.K.G.V. Lakmali Koralage (SIS/24/B2/13)
- G.B. Darsha Anuradha (SIS/24/B2/15) - Leader
- B.W.S.S. Nawarathna (SIS/24/B2/39)

1. Executive Summary & Domain Selection

- **Target Domain:** Healthcare & Bio-Medical Informatics.
- **Focus Area:** Digital Health & Decarbonization (Climate surveillance integrated with clinical data).
- **Overview:** The Medical Room Automation System (MRAS) is a Python-based, database-driven web application designed to digitize corporate medical room operations. It transitions facilities from inefficient paper logbooks to a secure digital ecosystem that manages patient histories, automated pharmacy inventory, and integrates environmental surveillance to correlate local air quality metrics with respiratory-related clinical data.

2. Situational Analysis & Gap Identification

Following a structured situational analysis framework, our team identified critical operational gaps within current corporate medical workflows:

- **Desk Review:** Many corporate medical rooms still rely on disparate manual processes. Patient data is recorded in physical logbooks, and medicine stock is calculated via periodic manual counts rather than real-time consumption tracking.
- **Stakeholder mapping:** Pharmacists/Management experience unexpected stock-outs of specific medications (e.g., inhalers, antihistamines) during seasonal or environmental changes due to inventory blind spots and a lack of predictive environmental data.
- **The Five Whys (Root Cause Analysis):**
    - *Why did the clinic run out of critical respiratory medicine?* The pharmacist was unaware that a spike in demand was imminent.
    - *Why was there a sudden spike in demand?* Because local air quality (AQI) dropped significantly over the last 48 hours, triggering asthma and allergy cases.
    - *Why was the clinic unprepared for the environmental impact?* Because clinical inventory and consultation workflows operate in isolation from environmental risk factors.
    - *Why are they isolated?* Because the clinic relies on disconnected manual processes rather than an interoperable framework.
    - **Root Cause:** The absence of an integrated software solution that fuses environmental climate monitoring data with clinical incidence records to adjust medical resource allocation.

2.1 User Pain Points & Problem Statement

- **Process Pain Point:** Manual data entry leads to duplicate patient profiles, illegible health histories, and significant time wasted on paperwork.
- **Financial/Security Pain Point:** Lack of real-time inventory tracking leads to expired medicine wastage, stock-outs, and an inability to audit who issued which medications.
- **Problem Statement:** Corporate medical facilities lack an integrated digital platform that fuses environmental risk factors with patient workflow, resulting in an inability to anticipate respiratory medicine shortages or correlate clinical data with local climate metrics.

3. Proposed Solution & Logic Decomposition

MRAS will be a professional-grade Python application resolving these pain points through four core modules:

1. **Digital Identity & Patient Module:** Replaces physical logs with unique digital patient profiles for instant retrieval and history tracking.
2. **Clinical Consultation Module:** A secure interface for doctors to log diagnoses and prescribe medication digitally, building a permanent health record.
3. **Automated Inventory Module:** A real-time pharmacy system that automatically deducts exact medicine quantities from the database immediately upon issuance, enforcing strict expiry date monitoring.
4. **Climate Surveillance Module:** Integrates a free external API to fetch current Air Quality Index (AQI) data/ other environment data, linking environmental risk factors directly to respiratory prescriptions.

3.1 Algorithmic Highlight: Dynamic Inventory Thresholds

To address the research gap, the backend business logic utilizes a dynamic multiplier to adjust inventory alerts based on real-time climate data:

1. **Baseline Verification:** The system queries the database for a medication's standard min_stock_level (e.g., Salbutamol Inhalers = 20 units).
2. **Climate Fetch:** The system queries the CLIMATE_LOG for the current daily AQI (e.g., AQI 155 - Unhealthy).
3. **Multiplier Application:** * AQI 0-50 (Good): Multiplier 1.0 (Threshold = 20)
    - AQI 51-100 (Moderate): Multiplier 1.2 (Threshold = 24)
    - AQI 101+ (Unhealthy): Multiplier 1.5 (Threshold = 30)
4. **Alert Trigger:** If current stock is 25, it exceeds the normal baseline but falls below the dynamic environmental threshold of 30. The dashboard instantly surfaces a predictive

warning: *"Warning: High AQI detected. Salbutamol stock is below the predictive safety threshold of 30!"*

4. Technical Architecture (Modular Monolith)

To align with industry standards and the module's requirements for maintainable code, MRAS will utilize a Modular Monolith architecture built on the Django Web Framework.

- **Interface Layer:** Django Templates styled with Bootstrap for responsive, accessible dashboards.
- **Domain/Business Logic Layer:** Python-based Django Views.
- **Persistence Layer:** SQLite utilized via Django's Object-Relational Mapper (ORM) to guarantee ACID-compliant transactions (e.g., deducting stock and saving a prescription simultaneously).

4.1 Modularity & Directory Structure

The application will enforce strict separation of concerns by isolating business domains into individual Django "apps":

```
mras_project/
│
├── manage.py                # Django's command-line utility
├── requirements.txt         # Pinned dependencies (Django, requests, selenium)
├── db.sqlite3               # The SQLite database (auto-generated)
├── .gitignore               # Prevents uploading __pycache__ and db.sqlite3 to GitHub
│
├── mras_core/               # THE PROJECT ROOT (Configuration & Dashboard)
│   ├── __init__.py
│   ├── settings.py          # Global settings, installed apps, database config
│   ├── urls.py              # Master URL routing (includes urls from other apps)
│   ├── views.py             # Holds the main Dashboard logic (combining API & Inventory)
│   ├── wsgi.py              # For server deployment (Render/Railway)
│   └── asgi.py
│
├── users/                   # MODULE 1: SECURITY & AUTH
│   ├── __init__.py
│   ├── admin.py             # Register models to view in Django Admin
│   ├── apps.py
│   ├── models.py            # Custom User model (Admin, Doctor, Pharmacist)
│   ├── urls.py              # Login/Logout routes
```

```
│   └── views.py            # Authentication logic
│
├── patients/               # MODULE 2: CLINICAL DOMAIN
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py             # Patient & Consultation database tables
│   ├── urls.py             # Patient registration and consultation routes
│   └── views.py            # Logic for saving patient records and diagnoses
│
├── inventory/              # MODULE 3: PHARMACY DOMAIN
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py             # Medicine, Inventory, and PrescriptionItem tables
│   ├── urls.py             # Routes for viewing stock and dispensing
│   └── views.py            # Logic for checking stock thresholds and deducting
│
├── climate/                # MODULE 4: ENVIRONMENTAL SURVEILLANCE
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py             # ClimateLog table for saving historical AQI
│   ├── urls.py             # (Optional) Routes if you want a dedicated climate page
│   └── views.py            # Logic containing the API fetch utility function
│
├── templates/              # GLOBAL FRONTEND TEMPLATES
│   ├── base.html             # The master HTML layout (Navbar, Sidebar, Bootstrap links)
│   ├── dashboard.html          # The main notification dashboard
│   ├── users/              # Login and profile templates
│   ├── patients/             # Consultation form templates
│   └── inventory/            # Stock table templates
│
└── static/                 # GLOBAL STATIC ASSETS
    ├── css/
    │   └── style.css       # Custom styling overriding Bootstrap
    ├── js/
    │   └── main.js         # Any custom frontend JavaScript
    └── images/             # Logos and icons
```

5. Database Schema & Entity-Relationship Design

The system's data integrity is maintained through a normalized relational database model. To enforce a strict separation of concerns, the tables are grouped by their respective Django applications.

MODULE 1: Security & Authentication (users app)

This table extends Django's default AbstractUser to handle Role-Based Access Control (RBAC).

Entity: USER	Data Type	Key / Constraint	Description
id	Integer	**Primary Key**	Unique identifier for the user.
username	String	Unique	Staff login ID.
password	String	Hashed	Securely hashed password.
role	String (20)	Choice	"Admin", "Doctor", or "Pharmacist".
is_active	Boolean	Default: True	Soft-delete flag for departed staff.

MODULE 2: Clinical Domain (patients app)

These tables manage the core medical workflow, linking patients to their diagnoses and the environmental conditions of that specific day.

Entity: PATIENT	Data Type	Key / Constraint	Description
id	Integer	**Primary Key**	Unique patient identifier.
employee_id	String (50)	Unique	Corporate ID for the patient.
full_name	String (255)	Not Null	Patient's complete name.
department	String (100)	Not Null	Corporate department of the employee.
date_of_birth	Date	Not Null	For age-based medical decisions.

Entity: CONSULTATION	Data Type	Key / Constraint	Description
id	Integer	**Primary Key**	Unique consultation record.
patient_id	Integer	**Foreign Key**	Links to PATIENT.id.
climate_log_id	Integer	**Foreign Key**	Links to CLIMATE_LOG.id (Nullable).
consultation_datetime	DateTime	Auto-Now	Exact time of the visit.
diagnosis	Text	Not Null	Doctor's clinical notes.

MODULE 3: Environmental Surveillance (climate app)

This table acts as a historical ledger for API data, preventing the system from making redundant API calls for every single consultation on the same day.

Entity: CLIMATE_LOG	Data Type	Key / Constraint	Description
id	Integer	**Primary Key**	Unique climate record.
log_date	Date	Unique	Only one record saved per day.
aqi_level	Integer	Not Null	The Air Quality Index (e.g., 150).
weather_condition	String (100)	Not Null	e.g., "Dust", "Clear", "High Humidity".
temperature	Float	Not Null	Daily temperature in Celsius.

MODULE 4: Pharmacy Domain (inventory app)

These tables handle the real-time stock deductions and link specific medicines to specific consultations.

Entity: MEDICINE	Data Type	Key / Constraint	Description
id	Integer	**Primary Key**	Unique medicine identifier.
name	String	Unique	Brand name (e.g., Ventolin).
generic_name	String	Not Null	Scientific name (e.g., Salbutamol).
min_stock_level	Integer	Default: 20	Threshold to trigger dashboard alerts.

Entity: INVENTORY	Data Type	Key / Constraint	Description
id	Integer	**Primary Key**	Unique batch identifier.
medicine_id	Integer	**Foreign Key**	Links to MEDICINE.id.
batch_number	String (50)	Unique	Tracking for pharmaceutical audits.
expiry_date	Date	Not Null	To prevent dispensing expired drugs.
current_stock	Integer	Check >= 0	Remaining units in this specific batch.

Entity: PRESCRIPTION_ITEM	Data Type	Key / Constraint	Description
id	Integer	**Primary Key**	Unique prescription line.
consultation_id	Integer	**Foreign Key**	Links to CONSULTATION.id.
medicine_id	Integer	**Foreign Key**	Links to MEDICINE.id.
quantity	Integer	Check > 0	Amount dispensed.
dosage_instructions	String (255)	Not Null	e.g., "2 puffs every 4 hours".

Entity-Relationship Mapping Summary

To ensure clear data flow without circular dependencies, the relationships are mapped as follows:

- **One-to-Many (1:N):**
  - One USER (Doctor) can have multiple CONSULTATION records.
  - One PATIENT can have multiple CONSULTATION records.
  - One CLIMATE_LOG can be linked to multiple CONSULTATION records (e.g., all patients who visited on a bad air quality day share the same climate record).
  - One CONSULTATION can have multiple PRESCRIPTION_ITEM records.
  - One MEDICINE can have multiple INVENTORY batches (to handle different expiry dates).

- One MEDICINE can be linked to multiple PRESCRIPTION_ITEM records.

6. Agile Delivery & Team Responsibilities

The project will be executed over a 4-week sprint using the Agile Scrum framework.

- L.B. Charith Jeewan (PM): Coordinates daily stand-ups, manages the Git repository and merges, tracks Sprint Goals.
- W.I.L. Withana (Domain Researcher): Validates situational analysis, sources realistic mock data for medicines and patients.
- G.B.D. Anuradha (Lead Backend): Implements Django models.py (Database schemas) and core business logic in views.py.
- B.W.S.S. Nawarathna (Lead Frontend): Develops Django HTML templates, UI/UX routing, and dashboard visualizations.
- H.K.G.V. Lakmali Koralage (QA & Docs): Develops Selenium scripts, executes EP/BVA testing, manages README.md, and drafts the final report.

6.1 Four-Week Sprint Schedule

- **Week 1 (Initiation):** Finalize Problem Statement, initialize GitHub repository, establish Django folder structure and virtual environments.
- **Week 2 (Execution - Logic):** Build SQLite database models, implement Patient Registration and Authentication routing.
- **Week 3 (Execution - Integration):** Implement Inventory deduction logic, integrate the free Weather API for AQI tracking, and execute initial cloud deployment (Render/Railway).
- **Week 4 (Closing & QA):** Execute formal testing lifecycle (Selenium, EP, BVA), finalize technical documentation, and verify live URL stability for presentation.

7. Testing & Validation Strategy

- **Test Design Techniques (Black-Box Testing):** We will utilize Equivalence Partitioning (EP) and Boundary Value Analysis (BVA) to rigorously validate input data and system constraints.
- **Boundary/Invalid Cases (Edge Cases):** Testing system behavior when a doctor attempts to prescribe exactly 0, negative amounts, or exactly 1 unit more than the maximum

available stock in the INVENTORY table. The system must successfully block the transaction and flag a validation error.

- **Automated UI Testing:** To validate critical user journeys efficiently, we will develop automated test scripts using Selenium WebDriver. Selenium will automate repetitive browser interactions, such as login workflows, form submissions, and navigating multiple windows/alerts.

8. Deployment & Mandatory Deliverables

- **Version Control:** Source code will be hosted on a public GitHub repository. The commit history will serve as evidence of equitable individual contribution.
- **Cloud Hosting:** The completed Django application and database will be deployed to a Render or Railway cloud server, ensuring it is accessible via a public URL for the final evaluation.
- **Documentation:** A comprehensive README.md and requirements.txt will be maintained at the repository root